project-proposal-formatter用户手册

给**中国式项目申报类文档**（项目建议书 / 申报书 / 任务书 / 投标书）套用特定机构排版规范的 Skill。

规范按「一类型一文件」组织在 `references/` 下。目前收录**国防科工局（SASTIND）**的两套：

文档类型	规范文件	版本
项目**建议书**	`references/format-proposal-sastind.md`	V4.4
项目**任务书**	`references/format-taskbook-sastind.md`	V1.4

这个 Skill 能做什么

输入一份排版混乱的草稿（`.docx`），输出一份套好规范的成品。处理范围包括：

- **字体字号**：正文宋体小四、标题层级各自字号
- **段落**：行距、首行缩进 2 字符、两端对齐、**关闭「与文档网格对齐」**（否则行距会被文档网格吸附，设了却不生效）
- **标题**：各级标题样式统一，并清理 run 级直接格式（否则会覆盖样式定义）
- **章节分页**：按文档类型分别处理——建议书=各一级章节从新页开始且不重复分页；任务书=**取消**一级标题分页（`w:pageBreakBefore` 一律移除），但保留承担页码体系的分节符
- **图表题注**：统一为 Word `Caption` 样式，识别并把手打的纯文本题注转成真正的 SEQ 域，编号顺延纠错
- **表格**：正文区表格居中 + 表内字号统一
- **目录**：主目录 + 图目录 + 表目录
- **页码**：前置区罗马数字、正文阿拉伯数字重新编号
- **异常处理**：章节漏号（Word 自动编号造成的假阳性）、异常字符

两套 SASTIND 规范的区别（选错文件会导致整篇字号体系错位）

	项目建议书	**项目任务书**
标题体系	**三级**，一级**四号不加粗**、二三级小四加粗	**四级**，一～四级**统一宋体小四 + 加粗**
正文行距	1.5 倍	**1.25 倍（表格除外、各级标题除外**，两者仍 1.5）
标题样式名	「建议书一/二/三级标题」	「任务书一/二/三/四级标题」
一级标题分页	**强制分页**（章节各起新页）	**不强制分页**，并清除已有的 `w:pageBreakBefore`
与文档网格对齐	全文取消（§ 1.1）	全文取消（§ 1.1）· **两文档一致**
其余	—	与建议书规范逐段一致

> 任务书规范是建议书规范 V4.3 的**复制派生**（两份都是自包含全文，可独立执行）。差异共 9 项（8 项外观 + 1 项分页行为），列在任务书文件开头的速查表里（第 10 项「网格对齐」登记为"同"）；**改了一份要**

> **评估是否同步另一份**—— 底本的通用机制（题注、分节、页码、网格对齐、校验法）应同步，**建议书专属口径（如强制分页）绝不能同步**。

目录结构

```
project-proposal-formatter/
├── SKILL.md                                  # 技能入口：触发条件、工作流、扩展方式
├── README.md
├── references/
│   ├── format-proposal-sastind.md            # 国防科工局 项目建议书 格式要求（V4.4）
│   └── format-taskbook-sastind.md            # 国防科工局 项目任务书 格式要求（V1.4）
└── scripts/
    ├── diagnose_docx.py                      # 离线诊断：规模统计/样式清单/漏号/异常字符/网格对齐
    ├── format_sastind_proposal.py            # 建议书格式调整参考实现（python-docx 离线路径）
    └── format_sastind_taskbook.py            # 任务书格式调整参考实现（python-docx 离线路径）
```

安装

把整个 `project-proposal-formatter/` 目录放进你的 Skill 目录：

- **用户级**（所有项目可用）：`~/.workbuddy/skills/`
- **项目级**（团队共享）：`<项目根>/.workbuddy/skills/`

```
git clone <本仓库地址> ~/.workbuddy/skills/project-proposal-formatter
```

基础大模型选择

建议使用：Deepseek-v4

依赖

- **Python 3** + `python-docx`（离线脚本路径用）

  ```
  pip install python-docx lxml
  ```

- **文档编辑通道**（可选，取决于宿主环境）：本 Skill 只负责「格式规范 + 编排流程」，实际文档编辑可交给宿主的本地文档编辑能力（如实时编辑器 MCP），也可直接用离线脚本。若你的环境没有这些，走 `scripts/` 的离线路径。

快速开始

```
# 1. 先诊断文档现状（建议书、任务书通用）
python scripts/diagnose_docx.py 你的文档.docx

# 2a. 套用「建议书」规范
python scripts/format_sastind_proposal.py 你的建议书.docx 输出.docx

# 2b. 套用「任务书」规范
python scripts/format_sastind_taskbook.py 你的任务书.docx 输出.docx

# 调试：分步存档以二分定位问题（STAGES=1）
STAGES=1 python scripts/format_sastind_taskbook.py 输入.docx 输出.docx
```

> ⚠ **两个脚本都是「参考实现 / 起手模板」，不是开箱即用的通用工具**，都含稿件特定硬编码（样式名、编号与题注策略）。移植到新稿前，**先读脚本 docstring 顶部的「必须按稿调整」清单**，并用 `diagnose_docx.py` 核对实际样式名与结构。
>
> - `format_sastind_proposal.py` 对应**建议书**规范：三级标题、一级标题宋体四号不加粗、全文 1.5 倍行距。
> - `format_sastind_taskbook.py` 对应**任务书**规范：四级标题（新建 4 个样式）、一～四级统一宋体小四加粗、正文区 1.25 倍行距（表格·前置·题注保持 1.5）、**一级标题"清分页"（STEP 4）**、题注连续编号 + SEQ 域、主目录过期缓存清理。两个脚本都会在"全文打底行距"那一轮**同时关闭网格对齐**（`w:snapToGrid w:val="0"`）。
> - **别拿建议书脚本跑任务书**（或反之）——不仅字号体系会整体错位，**分页逻辑还会做反**：建议书脚本会主动给一级标题**补**分页，而任务书要的是**清**分页。

处理新机构 / 新文档类型

1. 在 `references/` 新建 `format-<文档类型>-<机构简称>.md`（**文档类型在前**，如 `format-application-nsfc.md`）。
2. 套用 `SKILL.md` 里的 reference frontmatter 模板，节数按实际要求增减，不要为凑模板硬写空节。
3. 转写用户给的格式要求文档时**逐段逐条核对**，并主动补全原文没写但实操必备的维度（前置内容、页码体系、章节分页、已有目录/题注处理、异常字符、工具能力边界）。
4. 把「原文没写、由推断补的」条目单独列为**「待用户确认」清单**放在文件末尾。

同一机构再加一种文档类型时，请用 `SKILL.md` 里的**「复制派生法」**：先 `cp` 旧规范保底（技术条款一字不丢），再在开头加「差异速查表 + 版本记录 + 底本沿革」，然后逐项外科式改写（正文条款 + § 13 流程 + 验收清单）。

详见 `SKILL.md` 的 *Extending the Skill* 一节。

已知限制

- **域（Field）刷新**：目录、题注编号、交叉引用都是 Word 域。`.docx` 保存时缓存结果不会自动更新，交付后需在 Word/WPS 中按 `Ctrl+A` → `F9` 刷新一次。
- **无 Word COM 的环境**：Linux + WPS 等环境下无法通过自动化接口刷新域，只能靠人工按 F9；同时 WPS 命令行无头转 PDF 在部分发行版上静默失败，无法据此自校验。
- **可选项默认跳过**：如「错别字自检」属可选项，默认不执行，需用户明确要求才跑。

- **行距"设了没生效"**：若文档在 `w:sectPr/w:docGrid` 里定义了行网格，段落还勾着「如果定义了文档网格，则与网格对齐」时，行距会被吸附到网格行的整数倍（1.25 倍与 1.5 倍可能落到同一档）。本 Skill 的规范（§ 1.1）要求**全文取消**该勾选，脚本已实现；手工操作时在 Word 里 `Ctrl+A` → 段落 → 取消勾选即可。
- **两份 SASTIND 规范是复制关系**：不是引用关系，一份有更新时需人工评估是否同步到另一份。

版本

- `format-proposal-sastind.md`：**V4.4**（V4.4 新增 § 1.1 网格对齐：全文取消「与文档网格对齐」，段落级 `w:snapToGrid w:val="0"`）
- `format-taskbook-sastind.md`：**V1.4**（派生自建议书 V4.3；V1.1 补入 3 条实测踩坑 + 配套任务书脚本；V1.2 取消一级标题强制分页并改为清除已有分页属性；V1.3 同步建议书 V4.4 的 § 1.1 网格对齐；V1.4 明确"1.25 只作用于正文文字段落、各级标题仍是 1.5"，并修掉"标题样式写 `w:line=300` / 段落实际 360"的规范-实现不一致）
- 变更记录见各文件顶部的元信息。

许可

未附加许可证文件。上传到 GitHub 前请按需补充 `LICENSE`（如 MIT / Apache-2.0）。

使用案例

1、告诉workbuddy,可以说:请帮我安装这个技能,下载地址为：https://github.com/123wszk/project-proposal-formatter.git

2、使用skill:



使用建议

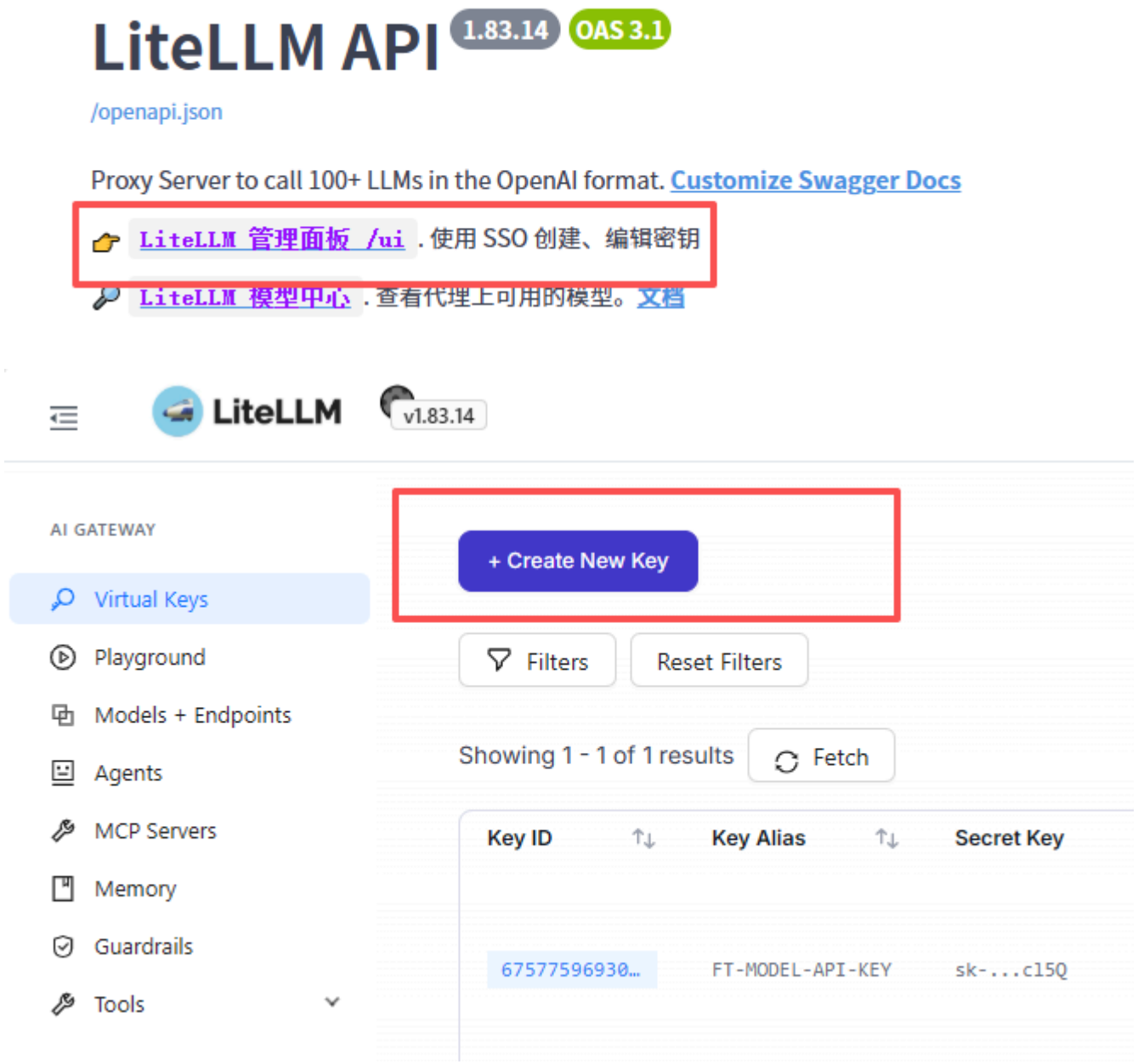
非涉密项目：建议采用workbuddy自带的基础大模型，相应速度较快

涉密项目：采用公司内部大模型，安全性较高，但相应速度慢

如何在workbuddy中配置使用公司内网的大模型?

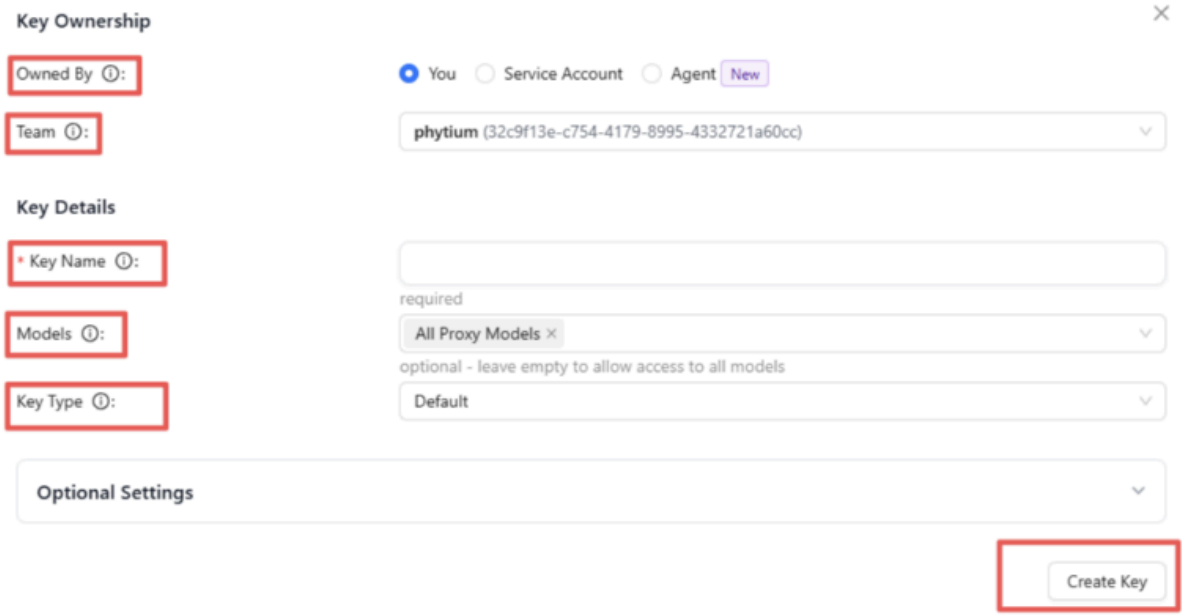
一、登录“https://aiapi.phytium.com.cn”申请API KEY

依次点击“API编程接口”——“LiteLLM 管理面板 /ui”——“create New Key”



注意：务必记录好自己的API KEY

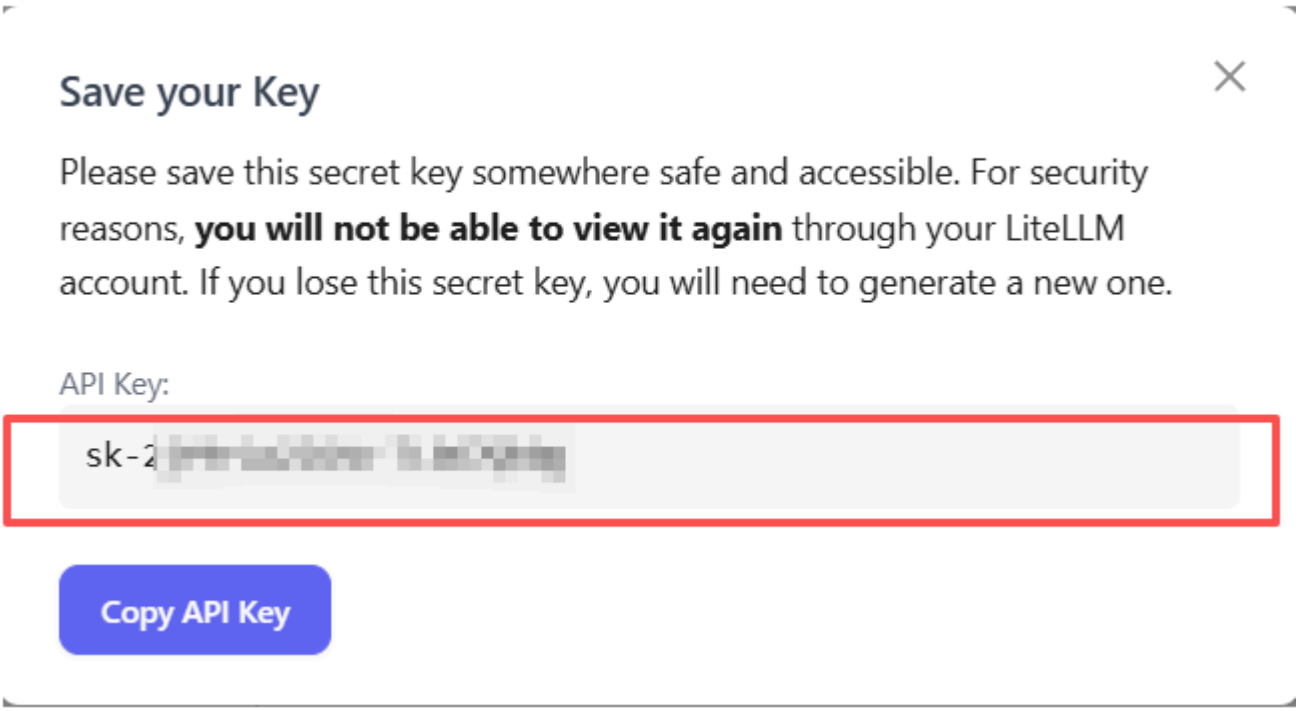
具体的API KEY设置，参考连接：https://aiclub.phytium.com.cn/pages/viewpage.action?pageId=148242523



配置参数说明：

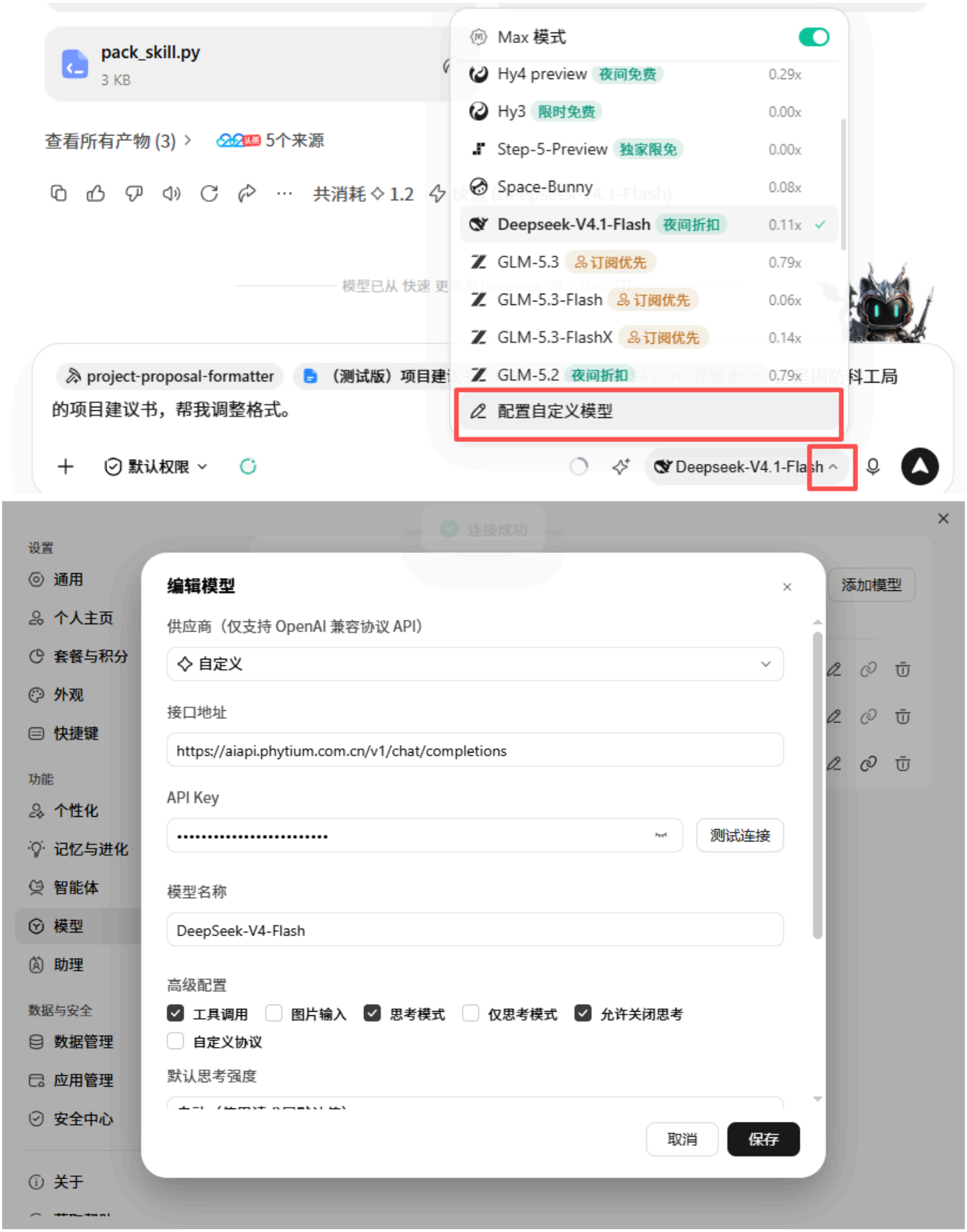
参数	说明	建议
Owned By	选择所有者	选择"You"
Team	选择团队	选择默认的"phytium"
Key Name	密钥名称	**全局唯一**，避免冲突
Models	可用模型	根据需要选择，可多选
Key Type	密钥类型	务必选择**Default**

> **重要提醒：配置完成后，点击Create Key按钮，此时会弹出对话框，切记保存对话框中的Key，此对话框是唯一且仅且一次显示。**

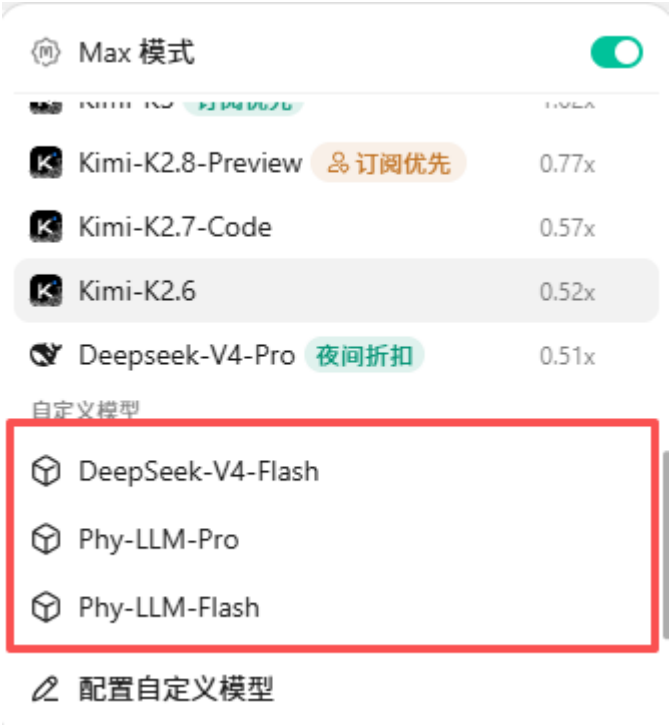


一、点击"配置自定义模型" 得到API KEY后，点击workbuddy中的"配置自定义模型"，接口地址为：https://aiapi.phytium.com.cn/v1/chat/completions

配置细节如下：



二、配置完成后，可在自定义模型中查看


Max 模式
Kimi-K2.8-Preview
订阅优先
0.77x
Kimi-K2.7-Code
0.57x
Kimi-K2.6
0.52x
Deepseek-V4-Pro
夜间折扣
0.51x
自定义模型
DeepSeek-V4-Flash
Phy-LLM-Pro
Phy-LLM-Flash
配置自定义模型